Terraform Task –2

Check for the terraform version and it creates the project folder

```
PS C:\Users\india> & "C:\Terraform\terraform.exe" version
Terraform v1.17.0
on windows_amd64

PS C:\Users\india> New-Item -ItemType Directory -Force "$HOME\terraform-02-practice"
>> Set-Location "$HOME\terraform-02-practice"
>> code .


    Directory: C:\Users\india


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        10/28/2026  11:57 AM                terraform-02-practice
```

Create main.tf file



This configuration tells Terraform to create a local file named `pets.txt` containing `My cat name is` Hen.

```
PS C:\Users\india\terraform-02-practice> terraform init
Initializing the backend...

Initializing provider plugins...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\india\terraform-02-practice>
```

Use terraform init

```
PS C:\Users\india\terraform-02-practice> terraform fmt
PS C:\Users\india\terraform-02-practice> terraform validate
Success! The configuration is valid.

PS C:\Users\india\terraform-02-practice> terraform plan

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
PS C:\Users\india\terraform-02-practice>
```

```
PS C:\Users\india\terraform-02-practice> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # local_file.my_pet will be created
  + resource "local_file" "my_pet" {
      + content              = "My cat name is Hen"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "pets.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value:
```

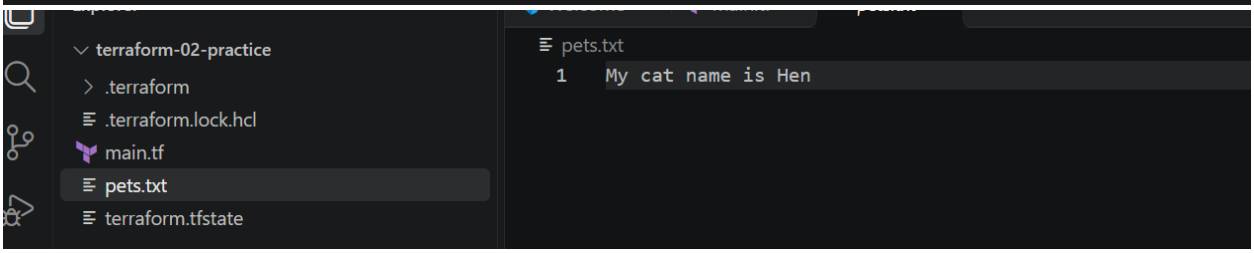
enter value yes

```
  Enter a value: yes

local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=90071a35e5befd871df1ec560dd47563dcad7811]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
PS C:\Users\india\terraform-02-practice>
```

```
PS C:\Users\india\terraform-02-practice> Get-Content .\pets.txt
My cat name is Hen
PS C:\Users\india\terraform-02-practice>
```



Change the file using below



Use the comand terraform init

```
Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Then terraform apply

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

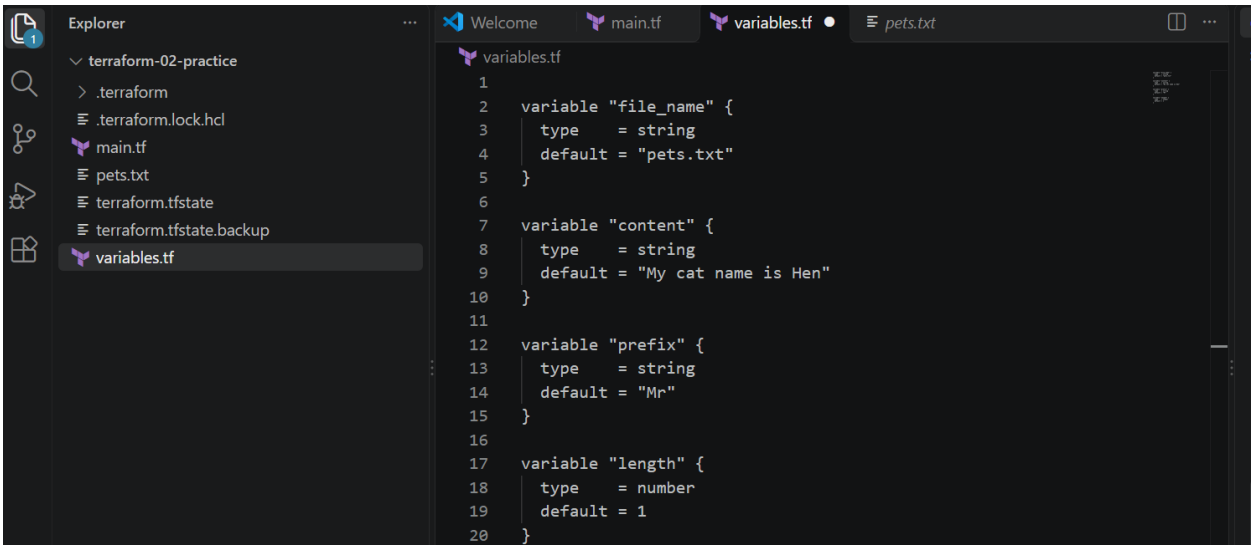
Outputs:

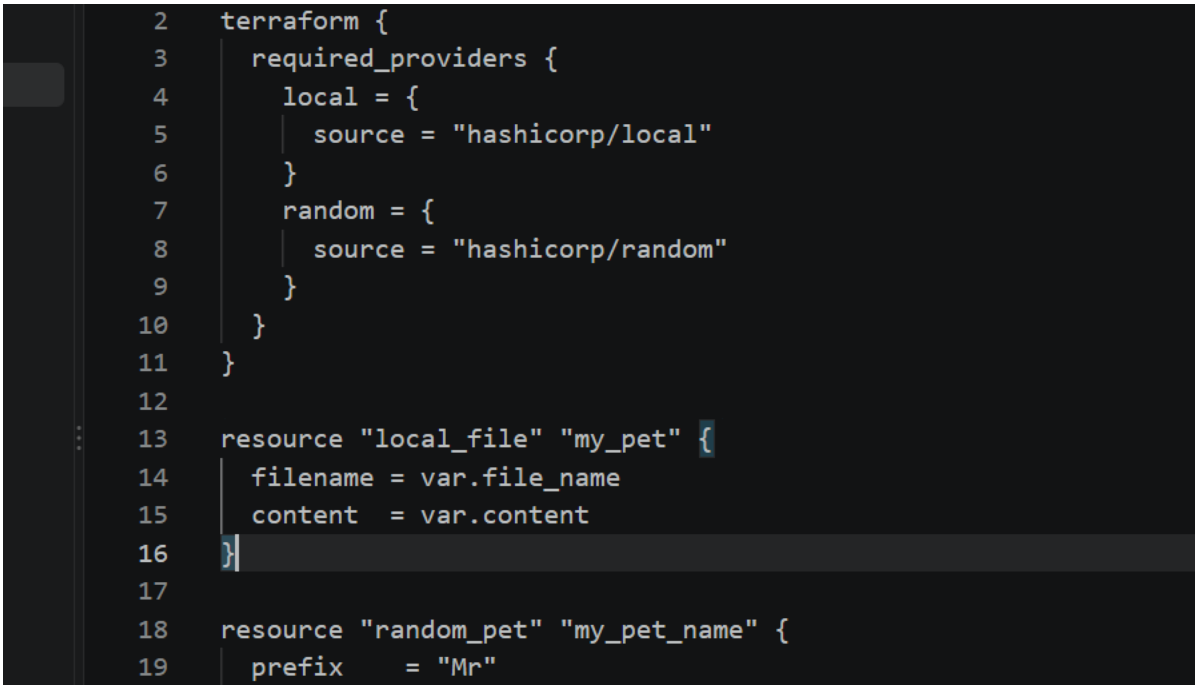
pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice>
```

Random provider is working, and Terraform generated the pet name `Mr.krill`.

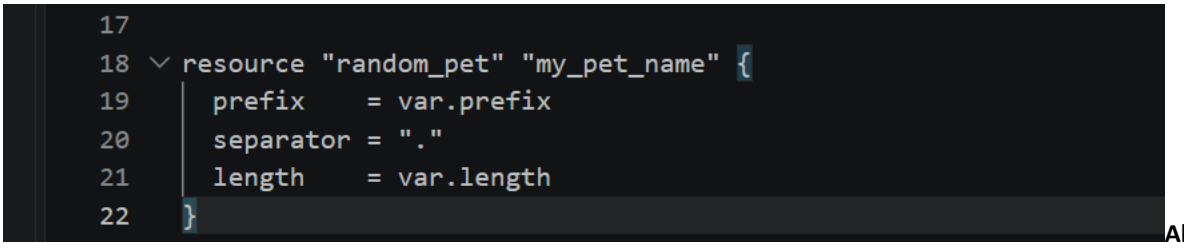
Variables.tf

Create variable.tf file and enter





Replace the file



Also change the random

```
variable "file_name" {
  type    = string
  default = "pets.txt"
}

variable "content" {
  type    = string
  default = "My cat name is Hen"
}

variable "prefix" {
  type    = string
  default = "Mr"
}

variable "length" {
  type    = number
  default = 1
}
```

Click the `variables.tf` tab at the top and make sure it contains all four variables: `file_name`, `content`, `prefix`, and `length`

```
pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice> terraform fmt
PS C:\Users\india\terraform-02-practice> terraform validate
Success! The configuration is valid.

PS C:\Users\india\terraform-02-practice> terraform plan
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
PS C:\Users\india\terraform-02-practice> terraform apply
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
```

```
Outputs:

pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice> terraform output pet_name
"Mr.krill"
PS C:\Users\india\terraform-02-practice> Get-Content .\pets.txt
My cat name is Hen
PS C:\Users\india\terraform-02-practice>
```

Create _before_destroy:

```
13
14  resource "local_file" "my_pet" {
15    filename = var.file_name
16    content  = var.content
17
18    lifecycle {
19      create_before_destroy = true
20    }
21  }
22
```

Replace the main file with above

`create_before_destroy = true` tells Terraform to create a replacement before destroying the old resource, when replacement is required.

This is useful for resources where downtime matters, although the behavior depends on the resource and whether both versions can coexist.

```
PS C:\Users\india\terraform-02-practice> terraform fmt
PS C:\Users\india\terraform-02-practice> terraform plan
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
PS C:\Users\india\terraform-02-practice> terraform apply
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice>
```

Prevent_destroy:

```
13
14  resource "local_file" "my_pet" {
15    filename = var.file_name
16    content  = var.content
17
18    lifecycle {
19      prevent_destroy = true
20    }
21  }
```

Replace lifecycle is true

```
PS C:\Users\india\terraform-02-practice> terraform fmt
PS C:\Users\india\terraform-02-practice> terraform plan
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
PS C:\Users\india\terraform-02-practice> terraform apply
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice>
```

Ignore_changes:

```
17
18    lifecycle {
19    ignore_changes = [content]
20    }
```

Replace the lifecycle

```
PS C:\Users\india\terraform-02-practice> terraform fmt
main.tf
PS C:\Users\india\terraform-02-practice> terraform plan
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.
PS C:\Users\india\terraform-02-practice> terraform apply
random_pet.my_pet_name: Refreshing state... [id=Mr.krill]
local_file.my_pet: Refreshing state... [id=90071a35e5befd871df1ec560dd47563dcad7811]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are
needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

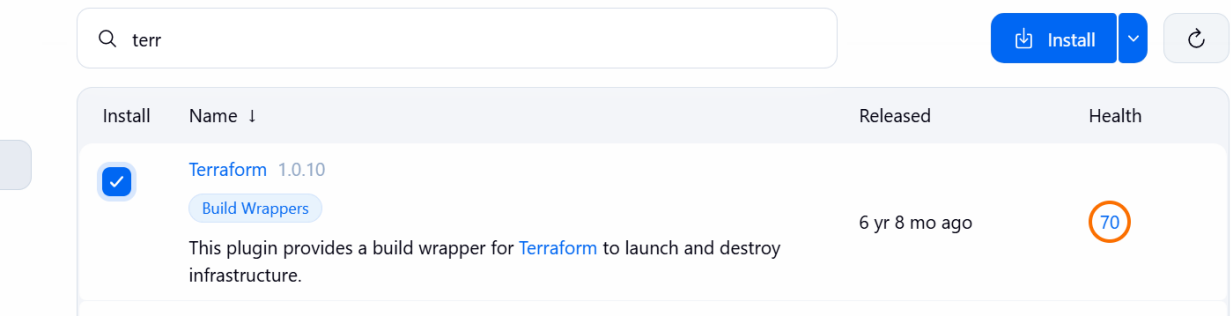
Outputs:

pet_name = "Mr.krill"
PS C:\Users\india\terraform-02-practice>
```

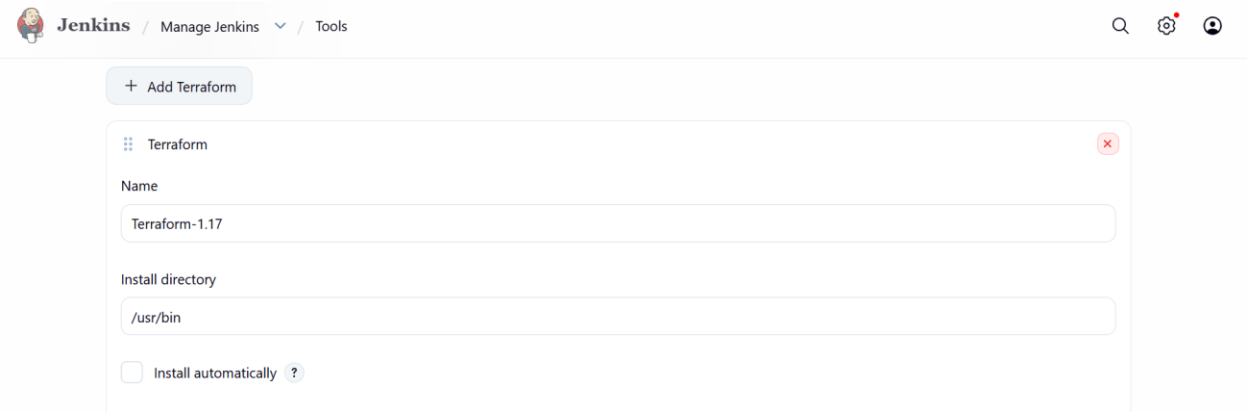
`create_before_destroy` — create a replacement before destroying the old resource when possible.

`prevent_destroy` — block planned destruction while the rule remains configured.

`ignore_changes` — ignore changes to selected attributes during updates.



Install the terraform plugin




```
[0m[1m[32mNo changes.[0m[1m Your infrastructure matches the configuration.[0m

[0mTerraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.
[0m[1m[32m
Apply complete! Resources: 0 added, 0 changed, 0 destroyed.[0m
Finished: SUCCESS
```

The build is success